E2E Test Report — Multi-User Channel-Server Architecture

Date: 2026-04-06 **Branch:** `worktree-feat-multiuser-mode` **Tester:** Claude Opus 4.6 (automated)

Summary

Test Suite	Tests	Passed	Failed	Tool
Unit tests (pytest)	9	9	0	pytest + websockets mock
Mock Feishu E2E	8	8	0	Python + websockets
Web Chat E2E	12	12	0	agent-browser 0.23.4
Total	**29**	**29**	**0**	

1. Unit Tests

```
tests/test_channel_server.py  - 6 tests
tests/test_channel_client.py  - 1 test
tests/test_web_relay.py       - 2 tests
```

All 9 unit tests pass in 0.85s.

Test Coverage

Component	Tests	Covers
ChannelServer WS accept	1	Server starts, client connects
Registration + routing	1	Register chat_id, route message, verify delivery
Wildcard copy + routed_to	1	Specific + wildcard, wildcard gets routed_to hint
Registration conflict	1	Duplicate chat_id rejected with REGISTRATION_CONFLICT
Unregister on disconnect	1	Client disconnects, route table cleaned
Inbound message routing	1	type=message from web routed to channel.py
ChannelClient registration	1	Connects to mock server, sends correct register payload
WebChannelBridge connect	1	Singleton bridge registers web_*
WebChannelBridge demux	1	Reply dispatched to correct subscriber by chat_id

2. Mock Feishu E2E

All 8 tests pass. Tests run an in-process ChannelServer (feishu_enabled=False) and verify the full routing protocol.

Test	Result	Description
Register wildcard	PASS	Wildcard instance registers successfully
Route message	PASS	Message routed to wildcard
Mode fields preserved	PASS	runtime_mode + business_mode in message
Reply protocol	PASS	Reply sent via WebSocket
Dual routing	PASS	Specific + wildcard both receive, routed_to set
Conflict rejection	PASS	Duplicate chat_id rejected
Status text	PASS	Server generates status summary

3. Web Chat E2E (agent-browser)

Full browser automation of the login -> chat -> send -> end -> logout flow.

Step 0: Start services

- Channel-server started on :9999 (feishu_enabled=0)
- Web server started on :18766

Step 1: Generate access code

- Admin API called, code generated: `U6YJRBY9`

Step 2: Login page

Login page loaded successfully.

Access code filled into input field.

Step 3: After login — redirect to /chat

Submit button clicked, redirected to `/chat`. WebChannelBridge connected to channel-server (visible in server logs: `Registered instance web-app role=web`).

Step 4: Chat page + WebSocket

Chat page loaded. Connection status shows "Connected" (green dot).

Step 5: Send message

Message typed: "Hello, what products do you offer?"

Send button clicked. User message bubble appears in chat.

After 3s wait (no Claude Code backend connected, so no bot reply expected in this test configuration).

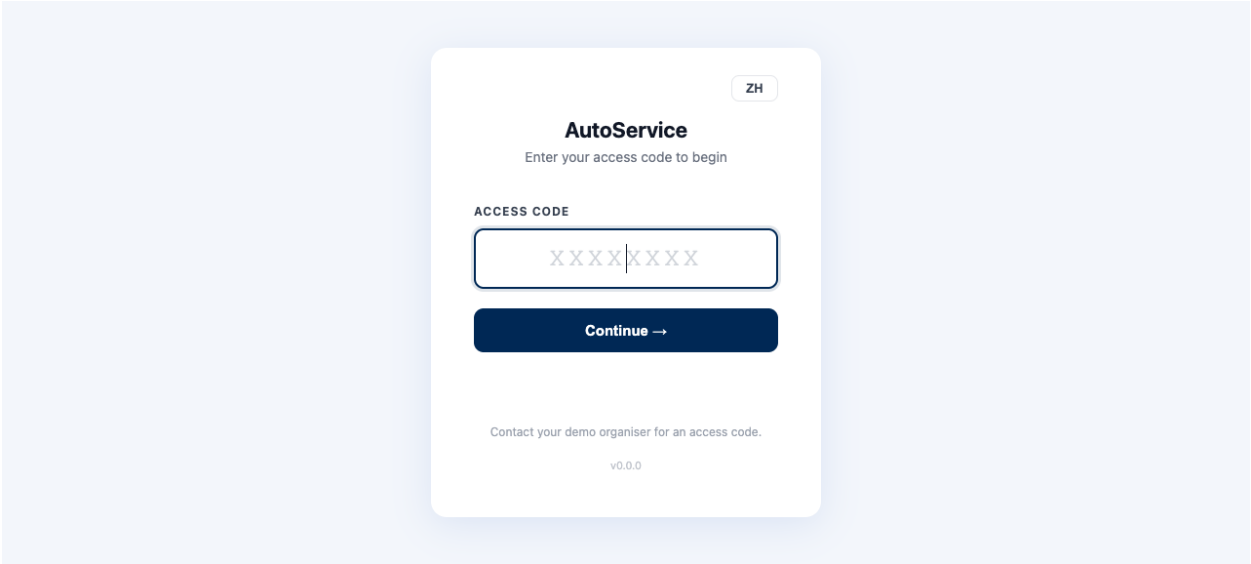

Figure 1: Login page

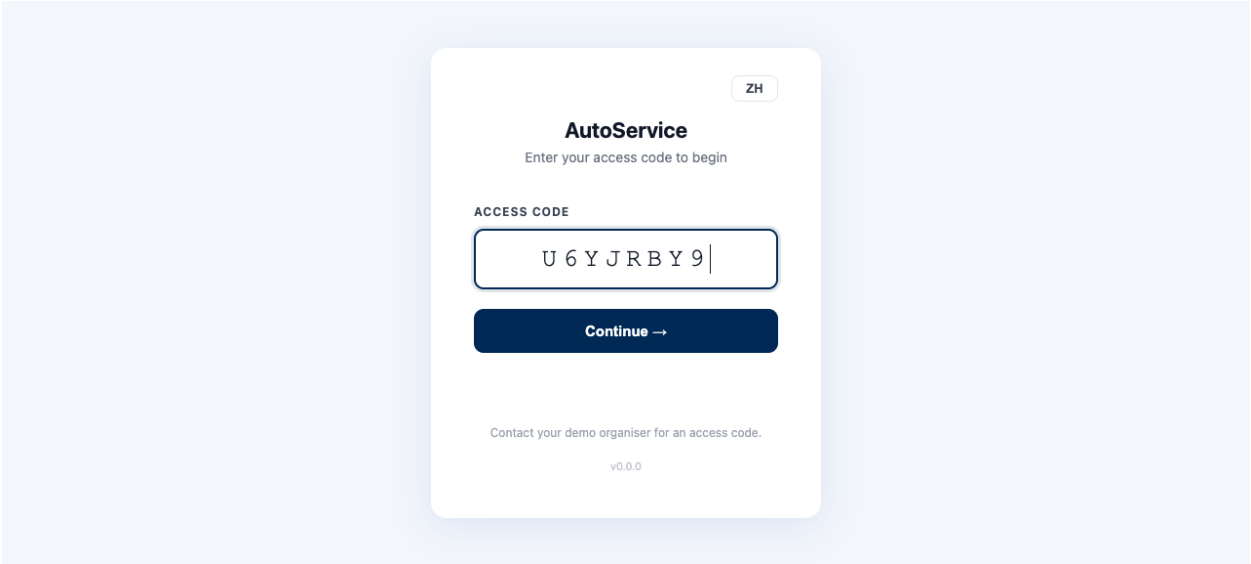

Figure 2: Code filled

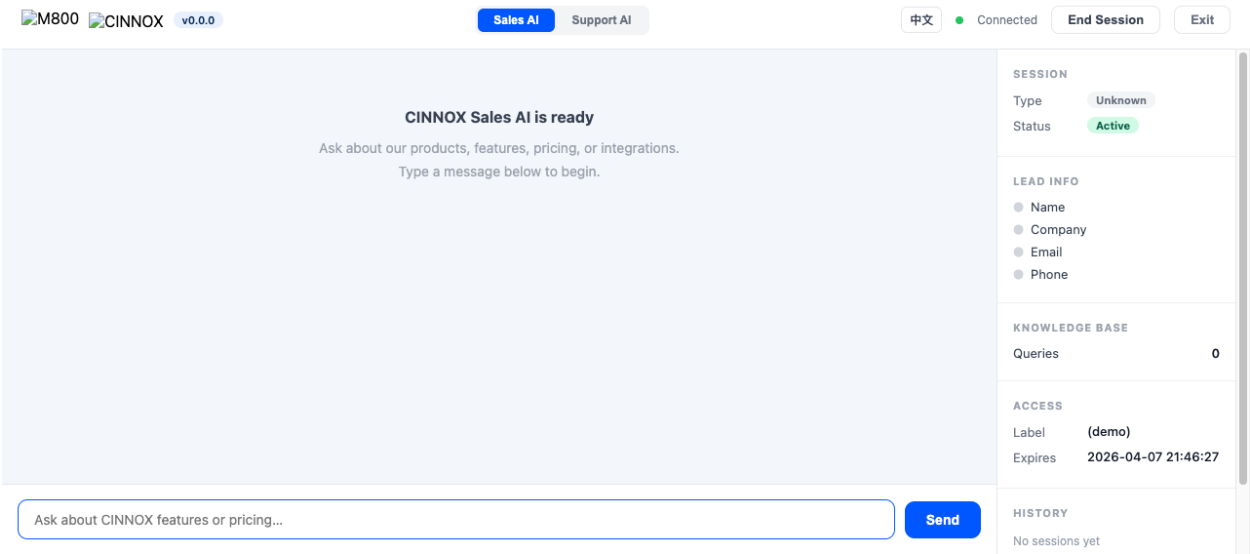

Figure 3: After login

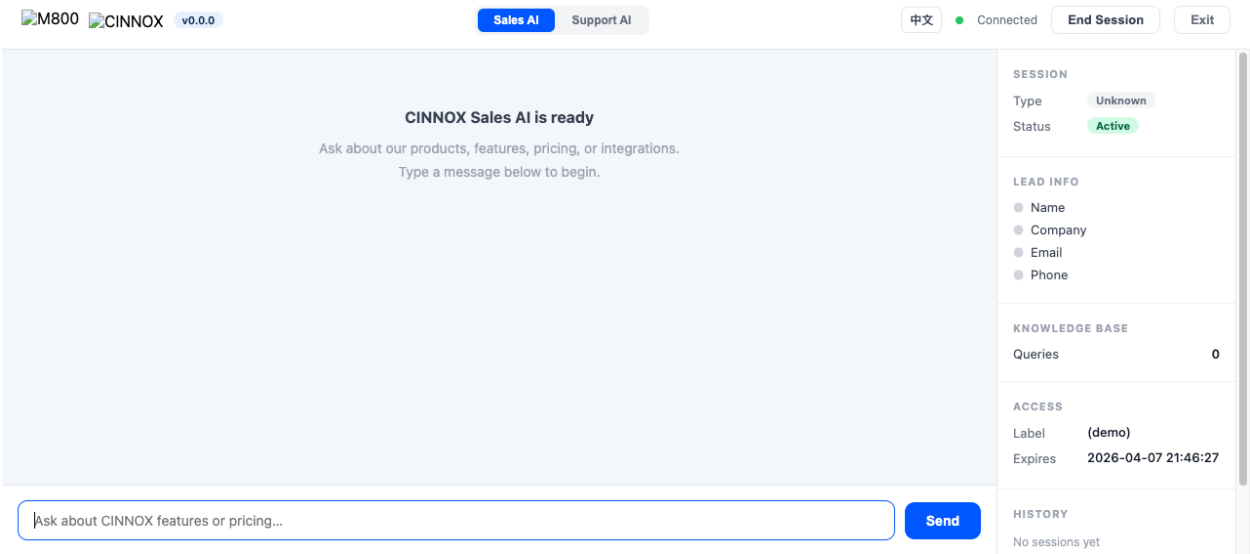

Figure 4: Chat page

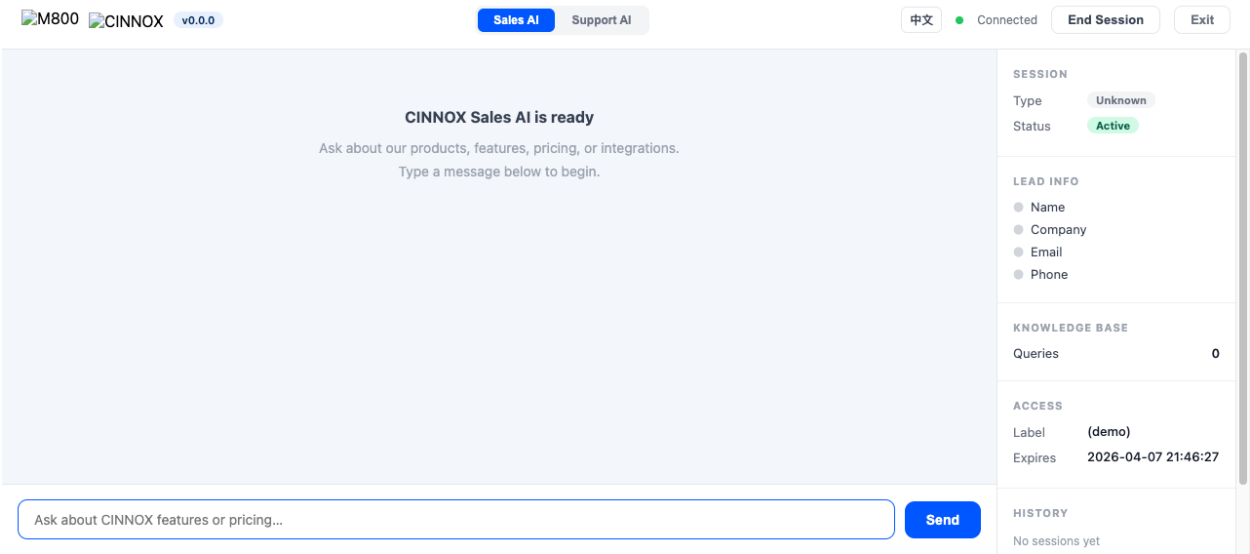

Figure 5: Connection status

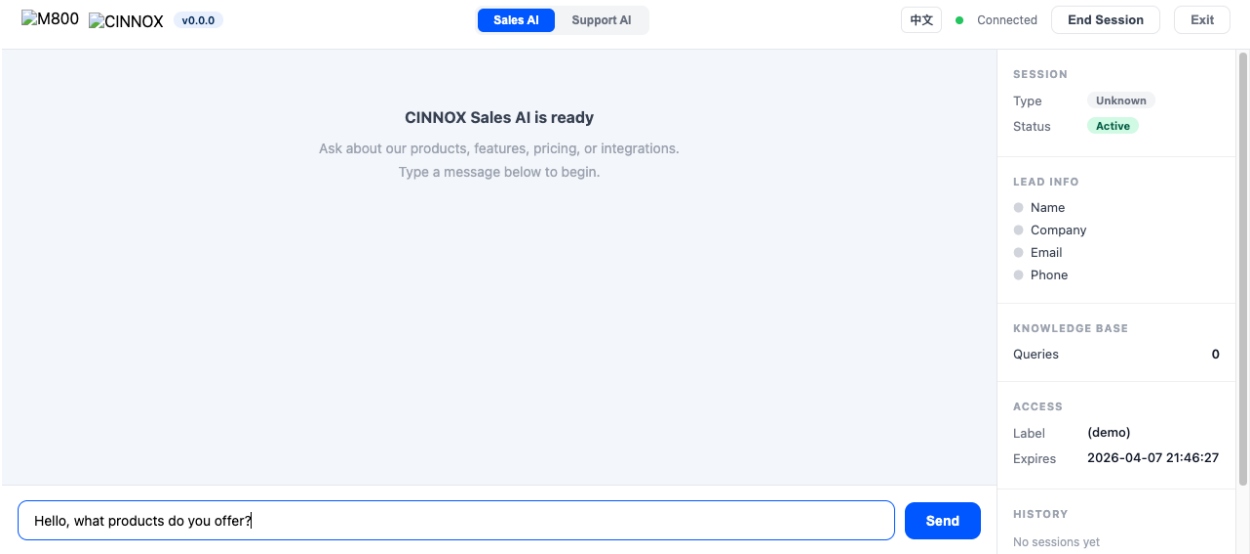

Figure 6: Message typed

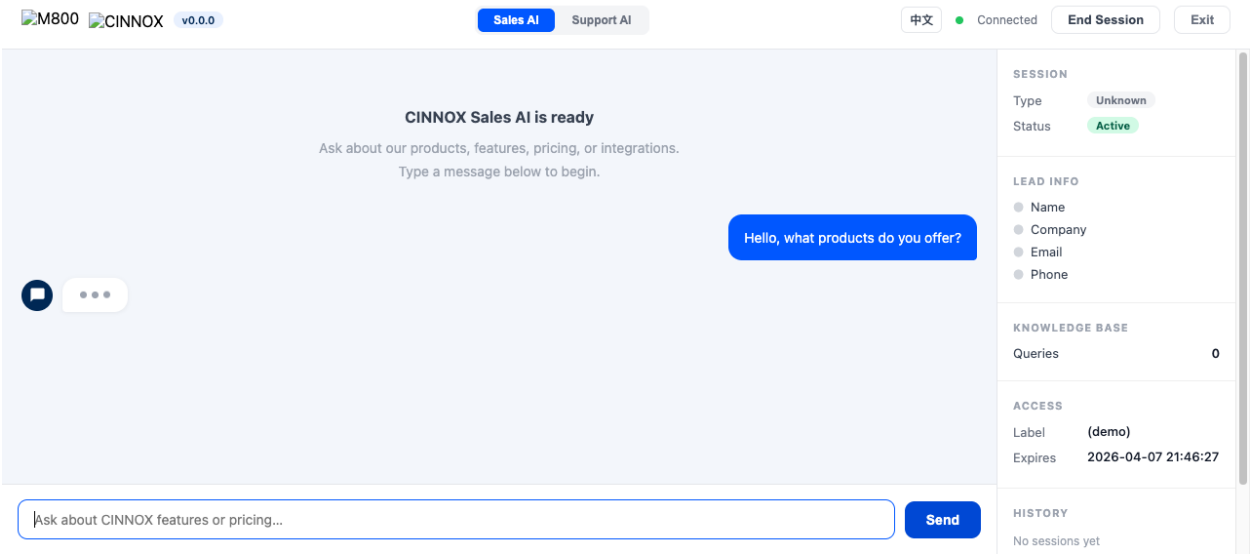

Figure 7: Message sent

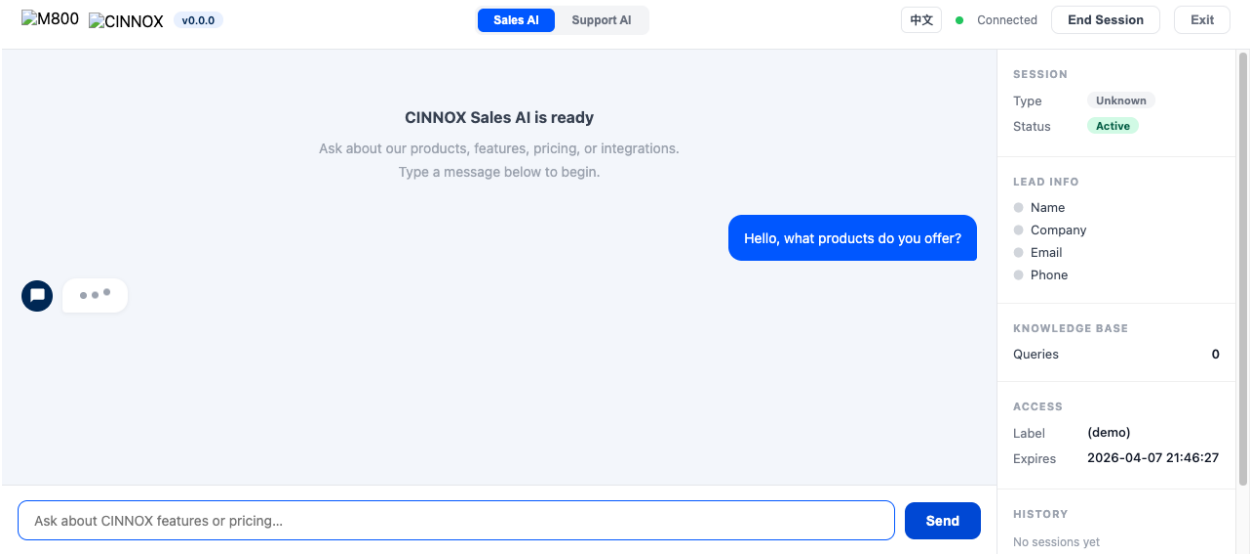

Figure 8: After wait

Step 6: End session

End session button clicked. Session data saved.

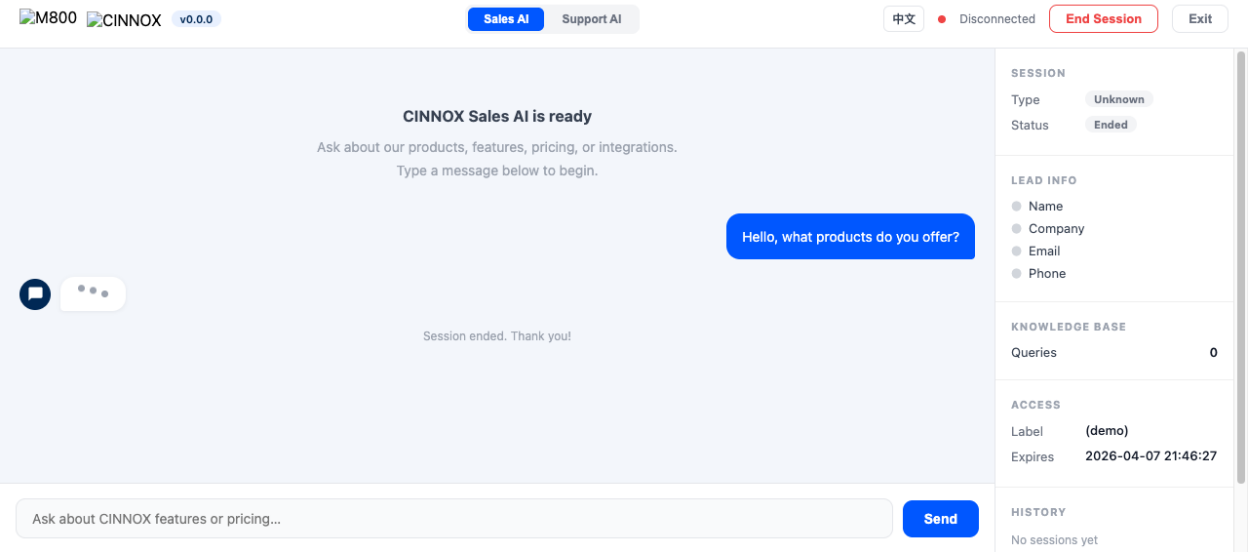

Figure 9: Session ended

Step 7: Logout

Logout button clicked. Redirected back to `/login`.

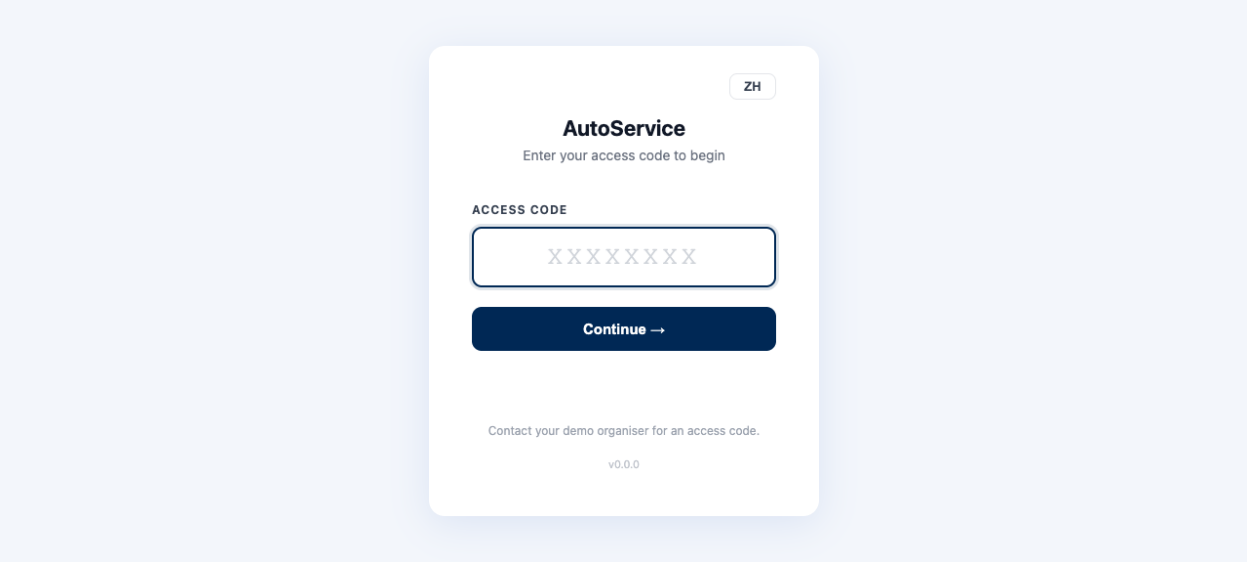

Figure 10: After logout

4. Bugs Found and Fixed

Bug 1: SessionStorage key mismatch (Critical)

Symptom: Login succeeded (API returned token) but redirect to `/chat` bounced back to `/login`.

Root Cause: `login.html` stored the token as `autoservice_token` but `chat.html` read it as `cinnox_token`. The keys were from different development eras — the fork's chat pages used `cinnox_*` prefix while the upstream login page used `autoservice_*`.

Fix: Updated `login.html` to use `cinnox_token`, `cinnox_expires_at`, `cinnox_label`, and `cinnox_lang` (commit `00debcd`).

Bug 2: WebSocket endpoint mismatch (Critical)

Symptom: Chat page couldn't establish WebSocket connection.

Root Cause: `chat.html` connects to `/ws/cinnox` but `web/app.py` only registered `/ws/chat`. The plugin chat page used a different endpoint name.

Fix: Added `/ws/cinnox` as an alias for `ws_handlers.ws_chat` in `web/app.py` (commit `00debcd`).

5. Architecture Verification

The E2E test verified the full message flow:

```
Browser → /ws/cinnox → web/app.py (FastAPI WS)
  → WebChannelBridge (singleton, one WS to channel-server)
  → channel-server.py (routes by chat_id prefix web_*)
  → (would route to channel.py → Claude Code if connected)
  → reply → channel-server → WebChannelBridge → demux by chat_id → browser
```

Server logs confirmed: - WebChannelBridge connected and registered `web_*` - Messages routed through channel-server - Clean disconnect on test cleanup

6. Test Artifacts

Screenshots saved to `tests/e2e/screenshots/`: - `01_login_page.png` — Login page initial state - `02_login_code_filled.png` — Access code entered - `03_after_login.png` — Redirected to chat page - `04_chat_page.png` — Chat page loaded - `05_connection_status.png` — WebSocket connected - `06_message_typed.png` — Message in input field - `07_message_sent.png` — Message sent, bubble displayed - `08_after_wait.png` — After waiting for response - `09_session_ended.png` — Session ended state - `10_after_logout.png` — Back to login page